

TECHNOLOGY STACK JUSTIFICATION

SHOP-A-LYTICS

TECHNOLOGY STACK AND JUSTIFICATION

ITBA
Prepared for the Shop-A-Lytics Product Team

VERSION 1.0

LAST REVISED 3 OCTOBER 2026

VERSION HISTORY				
VERSION	APPROVED BY	REVISION DATE	DESCRIPTION OF CHANGE	AUTHOR
1.0	Pending	03/10/2026	Initial draft: justification of the technology stack against the project brief's criteria, replacing the stack listed in BRD v1.1.	ITBA (Development Team)

1. PURPOSE

The project brief lets the group choose its own technology stack, provided it is free and is justified in writing on four criteria, which is team skills, fit for purpose, ecosystem maturity, and feasibility within the project timeframe. This document records that justification and the alternatives that were rejected.

It replaces the stack proposed in BRD Version 1.1, Section 11 (Django, PostgreSQL, Bootstrap 5 and Plotly). BRD Version 1.2 will record the change.

2. THE STACK

Layer	Choice
Front end	React (built with Vite) and Recharts for charts
Back end	Python with Flask
Data and analytics	pandas
Database	SQLite accessed through SQLAlchemy, as two separate databases (BoschCard analytics data, and application data)
Login and security	Flask sessions and Werkzeug password hashing
PDF export	fpdf2
Testing	pytest

3. JUSTIFICATION AGAINST THE BRIEF'S CRITERIA

TEAM SKILLS

- The team already knows React, given prior introduction to React-Admin and Java Script in previous years.*
- All members used Python this year. Flask, pandas, SQLAlchemy, fpdf2 and pytest are all Python, so one language covers the API, the analytics and the tests.*
- The data tools (generator, validator and loaders) are also Python, so the data layer and the API share one language and one set of skills.*
- No component requires a language or paradigm the team has not already used.*

FIT FOR PURPOSE

- Front end: React's component model suits a dashboard with month, week and day drill-down (RC 1), a chart-type switcher and shared state. Recharts is built for React and provides the bar, line and scatter charts the client asked for.*
- Back end: Flask is a small framework that serves the JSON API the React pages need, without a template system or large conventions to learn.*
- Analytics: pandas is the best suited tool for importing messy data, computing monthly totals and historical baselines, and implementing the anomaly rule (RC 7, RC 8).*
- Database: the data is relational (Student, Vendor_Type, Vendor, Transaction), so a SQL database fits, and SQLite satisfies the brief's persistence requirement. Two separate SQLite files keep BoschCard data apart from account data, as NFR 9 requires. SQLite has no native point or decimal type, so latitude and longitude are stored as numbers and money as integer cents.*

- Login: two roles (Manager and Member) with one organisation per user need nothing beyond Flask sessions and Werkzeug's salted password hashing (RC 6).
- PDF export: fpdf2 produces the tabular reports required for RC 4 without system dependencies.
- Testing: pytest lets the same tests run against several datasets, which protects the late data swap (NFR 8) and provides concrete evidence of the privacy mechanisms (NFR 2 to NFR 4).

ECOSYSTEM MATURITY

- React, Vite, Flask, pandas, SQLAlchemy, SQLite and pytest are widely used, extensively documented and long established, with large communities and plentiful learning material.
- Recharts and fpdf2 are smaller but actively maintained and sufficient for the stated requirements.

FEASIBILITY WITHIN THE TIMEFRAME

- The project has roughly four build weeks (BRD Section 12), and the lecturer's dataset arrives about two days before the 3 November deadline.
- A Flask and React skeleton, the data generator, the validator, the privacy-safe loader and twelve data-layer tests already exist.
- SQLite needs no database server, so a stranger can run the system from the README alone, as the brief requires.
- SQLAlchemy keeps a later move to PostgreSQL possible without rewriting queries.
- A column-settings file and a validation step absorb differences in the lecturer's data without code changes.

4. ALTERNATIVES CONSIDERED

Option	Why it was not chosen
Django	A larger framework with its own template system, which overlaps with React, and a steeper learning curve. Flask covers the same needs with less. (Proposed in BRD v1.1.)
Streamlit	Fastest for simple dashboards, but weak for a multi-page application with custom login, roles, a Team page and a designed interface.
Node.js and Express	The data tools and analytics are in Python; a Node back end would add a second back-end language.
FastAPI	Adds asynchronous programming and data-model concepts that a project of this size does not need.
PostgreSQL	Provides native point and decimal types but needs a database server, which makes reproducible setup harder. SQLite is sufficient at this scale. (Proposed in BRD v1.1.)

5. RISKS TO FEASIBILITY AND CONTROLS

Risk	Control
Two languages and two terminals (Python and JavaScript)	One README with exact commands, and a single script that builds the databases.
Python version too new for some packages	Agree one version (3.12 or 3.13 if installs fail), state it in the README and use it on every machine.
The lecturer's data differs from the data model (NFR 8)	Validator, column-settings file, start-up readiness check, rehearsal datasets and two timed swap rehearsals.

Risk	Control
SQLite allows one writer at a time	The analytics database is read-only; only the application database is written, by a handful of demo users.
A privacy mechanism is silently bypassed (NFR 2 to NFR 4)	All BoschCard queries go through one module, and privacy and vendor-isolation tests fail if a rule is skipped.
Scope grows past the deadline	Scope frozen at Session 3 (6 October 2026) and a feature freeze on 28 October.

6. FREE AND OPEN-SOURCE STATUS

Component	Licence
Python	PSF licence
Flask, Werkzeug	BSD
pandas	BSD
SQLite	Public domain
SQLAlchemy	MIT
React, Vite, Recharts	MIT
fpdf2	LGPL
pytest	MIT
Git	GPL v2

7. TEAM SKILLS EVIDENCE

Member	Role	Python	React / JS	SQL	Evidence
Nkazimulo	ITBA				
Iman	Project Manager				
Christiaan	Lead Developer				
Bourgeoise	QA Lead				